



Reporte de Análisis: Redes Neuronales para la Predicción de Seguros Médicos

Muñoz López Jairo Isaac 807-A



11 DE FEBRERO DE 2026

RNA

Universidad Tecnológica de la Mixteca

1. Introducción y Objetivo

El objetivo de este análisis es mostrar y comparar la eficiencia de diferentes funciones de activación (ReLU, Logistic, Tanh e Identity) en un Perceptrón Multicapa (MLP) para predecir costos de seguros médicos utilizando el dataset `insurance.csv`. Al tratarse de un problema de regresión, se ha utilizado la función `MLPRegressor`, perteneciente a la biblioteca `sklearn.neural_network`.

2. Metodología y Preprocesamiento

Las bibliotecas y funciones necesarias para realizar el problema de regresión fueron las siguientes:

```
import pandas as pd
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler,
LabelEncoder
from sklearn.metrics import mean_squared_error,
mean_absolute_error
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

La información fue obtenida de la página de kaggle proporcionada <https://www.kaggle.com/datasets/mosapabdelghany/medical-insurance-cost-dataset>, sin embargo por cuestiones de conexión wifi, fue necesario descargarla e importarla usando la función

```
dataset = pd.read_csv('insurance.csv')
```

Para garantizar que la red neuronal procese los datos correctamente, se realizaron algunas transformaciones, pues algunas variables son del tipo categóricas y fue necesario realizar una conversión numérica.

- **Codificación de variables:** Se utilizó `LabelEncoder` para variables binarias (`sex`, `smoker`). Ya que son categorías binarias y comparables, a continuación de muestra como se han realizado las conversiones

Para la característica `smoker`:

Yes: 1

No: 0

Para la característica sex:

Male: 1

Female: 0

Para la variable categórica `region` la conversión fue hecha con

```
dataset = pd.get_dummies(dataset, columns=['region'],
drop_first=True, dtype=int), ya que estas características no son comparables,
pues no podemos decir que southwest es mejor que
northwest por ejemplo, esta función asocia cada región de la categoría region con un
vector, en este caso hay que considerar que las característica región desaparece de
nuestro dataset y es remplazada por otras tres columnas(region_northwest,
region_southeast, region_southwest). La conversión fue la siguiente:
```

```
region_northwest: (1, 0, 0)
```

```
region_southeast: (0, 1, 0)
```

```
region_southwest: (0, 0, 1)
```

```
región_northeast: (0, 0, 0)
```

- **Estandarización:** Se aplicó `StandardScaler` a las características de entrada para que todas tuvieran media 0 y desviación estándar 1, facilitando la convergencia del optimizador `adam`.
- **Arquitectura:** Se propuso una red con dos capas ocultas de **100 y 6 neuronas** respectivamente, una tasa de aprendizaje de **0.01** y una semilla de aleatoriedad fija (`random_state=42`) para que se pueda reproducir.

3. Resultados Obtenidos

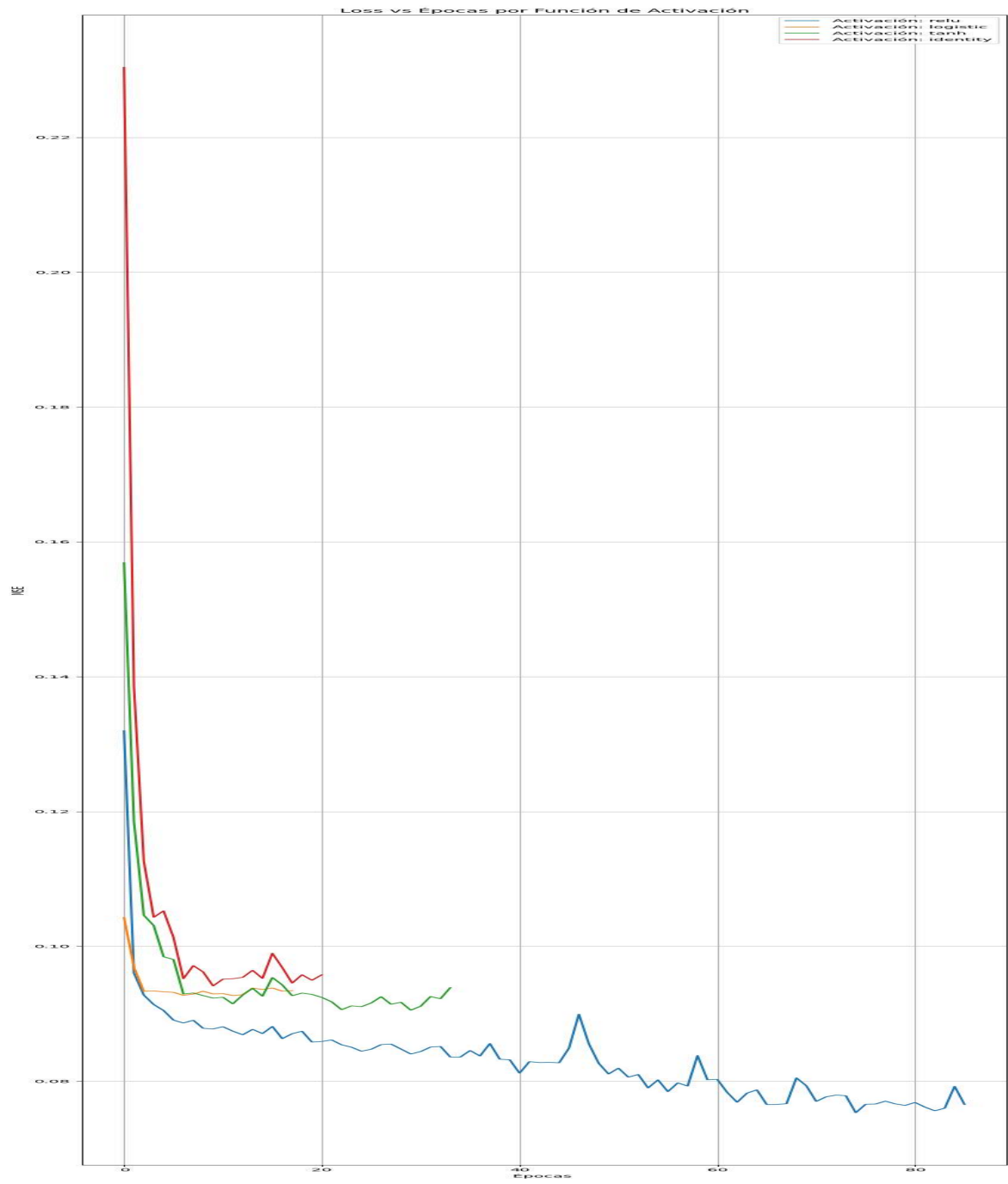
A continuación, se presentan las métricas de error tras 10,000 iteraciones (épocas):

Función de Activación	MSE	MAE
ReLU	0.196516	0.352414
Logistic	0.175579	0.348009

Tanh	0.188189	0.359863
Identity	0.176080	0.354210

4. Análisis de las Funciones de Activación

Para comparar que tan rápido aprende la red por cada tipo de función de activación, se realizó un gráfico en el que se mide el atributo `loss_curve_`. A continuación, se muestra el gráfico obtenido.



Basado en los resultados y la teoría de redes neuronales, se explican las diferencias:

- **ReLU (Rectified Linear Unit):** Es una función no lineal que activa neuronas solo si su valor es positivo. Aunque su error fue ligeramente superior, su curva de pérdida muestra una convergencia más rápida que el resto de funciones de activación.
- **Logistic (Sigmoid):** En cuestión de error, esta función obtuvo el **menor MSE**. Podría deberse al hecho de comprimir los valores entre 0 y 1 de forma suave. Fue la segunda con mejor convergencia.
- **Tanh (Tangente Hiperbólica):** En este caso tuvo el MAE más alto.
- **Identity:** Al ser una función lineal $f(x)=x$, no añade complejidad. Como los resultados son tan cercanos a los mostrados por la función `Logistic` podría indicarnos que el problema puede ser bien modelado de manera lineal.

5. Conclusión

La elección de la función de activación afecta directamente la velocidad de convergencia. Para este dataset específico, la función `Logistic` demostró ser la más eficiente en términos de precisión (MSE), aunque `ReLU` no puede ser tan fácilmente descartada porque muestra una convergencia similar, y además es eficiente computacionalmente. La predicción que podría arrojar el modelo es buena, aunque quizá se podría mejorar, tal vez manipulando la base de datos y eliminando los datos atípicos.

Para mostrar los resultados de manera organizada y facilitar la comparación, fue necesario apoyarse de un diccionario y un bucle en el que se entrena el modelo usando la función de activación correspondiente.

A continuación, se adjunta el código de Python correspondiente al problema.

```
import pandas as pd
```

```

from sklearn.neural_network import MLPRegressor

from sklearn.preprocessing import StandardScaler,
LabelEncoder

from sklearn.metrics import mean_squared_error,
mean_absolute_error

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

# Cargamos el dataset

dataset =
pd.read_csv('C:\\Users\\munlo\\Desktop\\python\\RNAClases\\in
surance.csv')

# Convertimos las variables categóricas a numéricas

le = LabelEncoder()

dataset['smoker'] = le.fit_transform(dataset['smoker'])
# smoker: si=1, no=0

dataset['sex'] = le.fit_transform(dataset['sex'])
# sex: male=1, female=0

'''la siguiente línea se explica mejor en el reporte, pero
básicamente se encarga de crear nuevas columnas para

cada región (norte, sur, este, oeste) y asignar un valor de 1
o 0 dependiendo de si la fila corresponde o no a

la región...'''

dataset = pd.get_dummies(dataset, columns=['region'],
drop_first=True, dtype=int) # asignamos variables numericas
a region y sex

print(dataset.filter(like='region')) #<-- esto es para
verificar que se hayan creado correctamente los vectores
correspondientes a cada región

```

```
# verificamos que se hayan convertido correctamente las
variables categóricas

X=dataset.iloc[:,0:6].values    #extraemos las primeras 6
columnas como características

Y=dataset.iloc[:, -1].values    #extraemos la última columna
como variable objetivo

X_train, X_test, y_train, y_test = train_test_split(X, Y,
test_size=0.2, random_state=42)    #dividimos el dataset en
entrenamiento y prueba

scaler = StandardScaler()    #estandarizamos las medidas para
mejorar el rendimiento de la red

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

#para comparar usaremos un bucle que itere sobre diferentes
funciones de activación y

# guarde los resultados en un diccionario para luego
presentarlos en una tabla

activaciones = ['relu', 'logistic', 'tanh', 'identity']
#funciones de activación a comparar

resultados = []    #aquí guardaremos los resultados de cada
función de activación

print("Comparación de funciones de activación:\n")
```



```

plt.figure(figsize=(10, 50))      # preparamos una figura para
graficar los resultados

for f in activaciones:

    # Definir el modelo con la función actual

    mlp = MLPRegressor(

        hidden_layer_sizes=(100, 6),

        activation=f,

        random_state=42,

        max_iter=10000,

        learning_rate_init=0.01,

        solver='adam'

    )

    # Entrenamos el modelo

    mlp.fit(X_train_scaled, y_train)

    # Predecimos con el conjunto de prueba

    y_pred = mlp.predict(X_test_scaled)

    # Error cuadrático medio

    mse = mean_squared_error(y_test, y_pred)

    # Error absoluto medio

    mae = mean_absolute_error(y_test, y_pred)

    # Guardar los resultados

    resultados.append({

        'Activación': f,

```

```

        'MSE': mse,

        'MAE': mae,

    })

    plt.plot(mlp.loss_curve_, label=f'Activación: {f}') #
    Graficamos la curva de pérdida para cada función de
    activación

    # mostramos resultados en una tabla
    df_resultados = pd.DataFrame(resultados)
    print("-" * 50)
    print(df_resultados.to_string(index=False))
    print("-" * 50)

    # Graficamos la comparación de las curvas de pérdida
    plt.title('Loss vs Épocas por Función de Activación')
    plt.xlabel('Épocas ')
    plt.ylabel('MSE')
    plt.legend()
    plt.grid(True)
    plt.savefig('curva_perdida_comparativa.png')
    plt.show()

```